

```

# -*- coding: utf-8 -*-
"""
ربات تلگرام نمایش کاتالوگ عطر
-----
میخونه و Excel یا CSV این ربات اطلاعات عطرها رو از یک فایل
به کاربر اجازه میده بر اساس برند جستجو کنه یا اسم عطر رو سرچ کنه.

نحوه اجرا:
1. pip install -r requirements.txt
2. ار بدید (یا مستقیم پایین کد ست کنید) BOT_TOKEN توکن ربات رو در متغیر محیطی
3. رو کنار این فایل بنارید catalog.xlsx یا catalog.csv
4. python bot.py
"""

import os
import logging
import pandas as pd
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import (
    Application,
    CommandHandler,
    CallbackQueryHandler,
    MessageHandler,
    ContextTypes,
    filters,
)

logging.basicConfig(
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
    level=logging.INFO,
)

logger = logging.getLogger(__name__)

# -----
# تنظیمات
# -----

BOT_TOKEN = os.environ.get("BOT_TOKEN", "PUT_YOUR_TOKEN_HERE")
CATALOG_PATH_CSV = os.path.join(os.path.dirname(__file__), "catalog.csv")
CATALOG_PATH_XLSX = os.path.join(os.path.dirname(__file__), "catalog.xlsx")

REQUIRED_COLUMNS = ["name", "brand", "notes", "description"]
OPTIONAL_COLUMNS = ["price", "image_url"]

PAGE_SIZE = 8 # تعداد آیتم در هر صفحه لیست

```

```

# -----
# بارگذاری کاتالوگ
# -----
def load_catalog() -> pd.DataFrame:
    if os.path.exists(CATALOG_PATH_XLSX):
        df = pd.read_excel(CATALOG_PATH_XLSX)
    elif os.path.exists(CATALOG_PATH_CSV):
        df = pd.read_csv(CATALOG_PATH_CSV)
    else:
        raise FileNotFoundError(
            "ید bot.py پیدا نشد. لطفاً آن را کنار catalog.xlsx یا catalog.csv فایل"
        )

    df.columns = [c.strip().lower() for c in df.columns]

    missing = [c for c in REQUIRED_COLUMNS if c not in df.columns]
    if missing:
        raise ValueError(f"ستون‌های زیر در فایل کاتالوگ وجود ندارند: {missing}")

    for col in OPTIONAL_COLUMNS:
        if col not in df.columns:
            df[col] = ""

    df = df.fillna("")
    df["id"] = df.index.astype(str)
    return df

CATALOG: pd.DataFrame = load_catalog()
logger.info("عطر بارگذاری شد %s کاتالوگ با", len(CATALOG))

def get_brands() -> list:
    return sorted([b for b in CATALOG["brand"].unique() if b])

# -----
# نمایش کارت یک عطر
# -----
def format_item(row: pd.Series) -> str:
    text = f"🌸 *{row['name']}*\n"
    text += f"👉 برند: {row['brand']}\n"
    if row["notes"]:
        text += f"🎵 نوت‌ها: {row['notes']}\n"
    if row["price"]:

```

```

        text += f"$ قیمت: {row['price']}\n"
    if row["description"]:
        text += f"\n{row['description']}\n"
    return text

async def send_item(update_or_query, row: pd.Series):
    text = format_item(row)
    image_url = row.get("image_url", "")
    target = update_or_query.message if hasattr(update_or_query, "message") else

    if image_url:
        try:
            await target.reply_photo(photo=image_url, caption=text, parse_mode="M
            return
        except Exception:
            logger.warning("ناموفق بود، فقط متن ارسال می‌شود %s ارسال عکس برای",

            await target.reply_text(text, parse_mode="Markdown")

# -----
# هندلرها
# -----
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    keyboard = [
        [InlineKeyboardButton("📖 مشاهده همه عطرها", callback_data="list_page_0")
         [InlineKeyboardButton("🏪 جستجو بر اساس برند", callback_data="brands")],
         [InlineKeyboardButton("🔍 جستجو با نام", callback_data="search_hint")],
    ]
    await update.message.reply_text(
        "یکی از گزینه‌های زیر رو انتخاب کنید🌹 سلام! به فروشگاه عطر خوش اومدید",
        reply_markup=InlineKeyboardMarkup(keyboard),
    )

async def show_list_page(query, page: int, df: pd.DataFrame, prefix: str):
    start_i = page * PAGE_SIZE
    end_i = start_i + PAGE_SIZE
    chunk = df.iloc[start_i:end_i]

    buttons = [
        [InlineKeyboardButton(f"{row['name']} - {row['brand']}", callback_data=f"
        for _, row in chunk.iterrows()
    ]

    nav_row = []

```

```

if start_i > 0:
    nav_row.append(InlineKeyboardButton("⬅ قبلی", callback_data=f"{prefix}_{
if end_i < len(df):
    nav_row.append(InlineKeyboardButton("➡ بعدی", callback_data=f"{prefix}_{
if nav_row:
    buttons.append(nav_row)

buttons.append([InlineKeyboardButton("🏠 بازگشت به منو", callback_data="home"

await query.edit_message_text(
    f"عطر {page + 1} - {len(df)} صفحه",
    reply_markup=InlineKeyboardMarkup(buttons),
)

async def button_handler(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    data = query.data

    if data == "home":
        keyboard = [
            [InlineKeyboardButton("📖 مشاهده همه عطرها", callback_data="list_page
            [InlineKeyboardButton("🏷 جستجو بر اساس برند", callback_data="brands"
            [InlineKeyboardButton("🔍 جستجو با نام", callback_data="search_hint")
        ]
        await query.edit_message_text(
            "یکی از گزینه‌های زیر رو انتخاب کنید",
            reply_markup=InlineKeyboardMarkup(keyboard),
        )

    elif data.startswith("list_page_"):
        page = int(data.split("_")[-1])
        await show_list_page(query, page, CATALOG, "list_page")

    elif data == "brands":
        brands = get_brands()
        buttons = [[InlineKeyboardButton(b, callback_data=f"brandsel_{b}")] for b
        buttons.append([InlineKeyboardButton("🏠 بازگشت", callback_data="home")])
        await query.edit_message_text("یک برند رو انتخاب کنید", reply_markup=Inl

    elif data.startswith("brandsel_"):
        brand = data.split("_", 1)[1]
        filtered = CATALOG[CATALOG["brand"] == brand].reset_index(drop=True)
        filtered["id"] = CATALOG[CATALOG["brand"] == brand].index.astype(str)
        context.user_data["brand_filter"] = brand
        await show_list_page(query, 0, filtered, f"brandpage_{brand}")

```

```

elif data.startswith("brandpage_"):
    parts = data.rsplit("_", 1)
    brand = parts[0].replace("brandpage_", "")
    page = int(parts[1])
    filtered = CATALOG[CATALOG["brand"] == brand]
    await show_list_page(query, page, filtered, f"brandpage_{brand}")

elif data == "search_hint":
    await query.edit_message_text(
        "اسم عطر مورد نظرتون رو تایپ کنید و برام بفرستید 🔍\n"
        "(رو بزنید /start برای بازگشت به منو دستور)"
    )

elif data.startswith("item_"):
    item_id = data.split("_", 1)[1]
    row = CATALOG.loc[CATALOG["id"] == item_id].iloc[0]
    await send_item(query, row)

async def text_search(update: Update, context: ContextTypes.DEFAULT_TYPE):
    q = update.message.text.strip().lower()
    results = CATALOG[
        CATALOG["name"].str.lower().str.contains(q)
        | CATALOG["brand"].str.lower().str.contains(q)
    ]

    if results.empty:
        await update.message.reply_text("دوباره امتحان کنید 😞")
        return

    for _, row in results.head(5).iterrows():
        await send_item(update, row)

    if len(results) > 5:
        await update.message.reply_text(f"{len(results) - 5} ی دقیقتری انجام بدید")

# -----
# اجرای ربات
# -----
def main():
    if BOT_TOKEN == "PUT_YOUR_TOKEN_HERE":
        raise RuntimeError("لطفا توکن ربات را در متغیر محیطی BOT_TOKEN دهید")

    app = Application.builder().token(BOT_TOKEN).build()

```

```
app.add_handler(CommandHandler("start", start))
app.add_handler(CallbackQueryHandler(button_handler))
app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, text_search))
```

```
logger.info("ربات در حال اجراست...")
app.run_polling()
```

```
if __name__ == "__main__":
    main()
```